Autorouter Technical Approaches and Architecture

https://gemini.google.com/app/512bbee6551f1b4a

User prompt: I want a deep research dive into autorouter technical approaches.

Response: Electronic Design Automation (EDA) autorouting is fundamentally an **NP-complete** combinatorial optimization problem. For a single multi-pin net, finding the shortest interconnect is the **Rectilinear Steiner Minimum Tree (RSMT)** problem; for a full board or die, it becomes the **Multi-Commodity Flow** and **Vertex-Disjoint Path** problem across a 3D manifold with non-uniform geometric constraints.

Because solving all nets simultaneously is computationally intractable at scale, production autorouters decompose the problem spatially (Global vs. Detailed routing), geometrically (Grid vs. Shape vs. Topological), and temporally (Sequential Rip-up vs. Negotiated Congestion).

1. Geometric & Spatial Paradigms

The core differentiator between autorouter architectures is how they represent empty board space, obstacles, and trace geometry.

Paradigm	Spatial Representation	Time / Space Complexity	Primary Strengths	Primary Failure Modes	Representative Engines
Grid-Based (Maze)	Uniform 2D/3D discrete voxel array	$\mathcal{O}(W \cdot H \cdot L)$ memory and search per net	Guaranteed shortest path for single net; simple data structures	Memory explosion on fine grids; fails on off-grid mixed-pitch pads (e.g., 0.4 mm BGA + imperial passives)	Classic Lee/Soukup routers, early Eagle/KiCad plugins
Line-Probe (Escape)	Continuous coordinates; orthogonal ray-casting	$\mathcal{O}(N_{\text{obstacles}})$ memory; fast heuristic search	Ultra-low memory footprint; fast initial long-distance connections	Non-optimal detour lengths; struggles in high-density obstacle mazes	Hightower, Mikami-Tabuchi
Shape-Based (Gridless)	Continuous polygons; R-trees / Kd-trees / Corner-stitching	$\mathcal{O}(V \log V)$ spatial queries (V = obstacle vertices)	Native support for arbitrary trace widths, clearances, and off-grid pads	Can trap itself in acute-angle or stair-step local optima without strong glossing	Cadence Specctra / Allegro, Freerouting, Altium Situs
Topological (Rubber-Band)	Constrained Delaunay Triangulation (CDT) mesh	$\mathcal{O}(P \log P)$ triangulation (P = pins/obstacles)	Decouples path homotopy from geometry; naturally yields smooth, any-angle paths	Difficult to enforce strict rectilinear/Manhattan bus structures or tight meander tuning	TopoR, FreeStyle, gEDA Toporouter, JITX Router

Grid-Based (Maze) Routing

Originating with **C.Y. Lee's algorithm (1961)**, maze routers discretize each copper layer into a uniform grid where cells are marked as free, blocked, or weighted by cost (e.g., penalizing wrong-way routing on a layer biased for horizontal traces).

- **Wave Propagation (Lee):** Performs a Breadth-First Search (BFS) radiating outward from source S until it hits target T, labeling each cell with its distance from S, then backtracing from T to S. In the heatmap below, the concentric color gradient shows the wavefront expanding around dark brown obstacle blocks from the source `S` at the bottom toward the target `T` at the top.

- **Heuristic Acceleration (A^* / Soukup / Hadlock):** Pure BFS explores uniformly in all directions. Modern grid routers replace BFS with A^* **search** using an octile or Manhattan distance heuristic:

  $$f(n) = g(n) + \epsilon \cdot h(n)$$

  where $g(n)$ is the accumulated path and via cost from the source, $h(n)$ is the Manhattan lower bound to the target, and $\epsilon \geq 1$ is an inflation factor trading path optimality for search speed. **Hadlock's algorithm** further optimizes memory by tracking "detour numbers" (the number of grid steps moving away from the target) rather than storing full distance integers in every cell.

Shape-Based (Gridless) Routing

Grid routers break down when a board combines metric fine-pitch BGAs (e.g., 0.35 mm pitch), imperial connectors (2.54 mm), and varying net-class clearances (e.g., high-voltage creepage next to 50 Ω logic). A uniform grid fine enough to resolve the greatest common divisor of all rules consumes gigabytes of RAM.

- **Minkowski Sum Expansion:** Shape-based routers (pioneered by Cooper and Chyan's **Specctra**) treat pads, vias, and existing traces as exact geometric polygons in continuous space. To route a trace of width W_{trace} with clearance C_{rule} past an obstacle polygon O, the router computes the **configuration space obstacle** via the Minkowski sum:

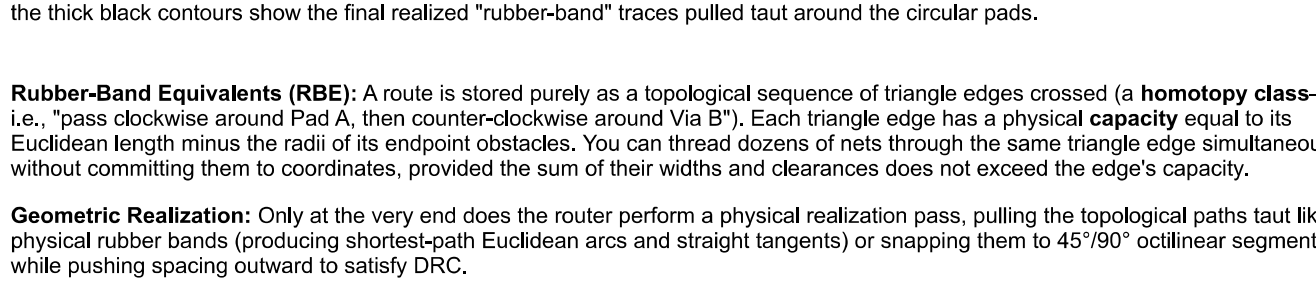
$$O_{\text{expanded}} = O \oplus B\left(\frac{W_{\text{trace}}}{2} + C_{\text{rule}}\right)$$

where $B(r)$ is a disc or octagon of radius r. The routing problem is thus reduced to routing a zero-width line along the visibility graph or free-space rectangles (often managed via **Ousterhout's Corner-Stitching** data structure or dynamic **R-trees**) between expanded obstacles.

Topological (Rubber-Band) Routing

Grid and shape-based routers lock a trace into specific (x, y) coordinates the moment it is placed, meaning an early trace running down the middle of a channel blocks subsequent traces even if there is technically room for both.

- **Homotopy & Triangulation:** Topological routers discard exact geometry during the search phase. They partition the board's obstacles (pads, pins, keepouts) using a **Constrained Delaunay Triangulation (CDT)**. In the diagram below, the gray background lines form the Delaunay triangulation mesh between pin nodes, the colored segments show the topological paths threading through triangle edges, and the thick black contours show the final realized "rubber-band" traces pulled taut around the circular pads.



- **Rubber-Band Equivalents (RBE):** A route is stored purely as a topological sequence of triangle edges crossed (a **homotopy class**—i.e., "pass clockwise around Pad A, then counter-clockwise around Via B"). Each triangle edge has a physical **capacity** equal to its Euclidean length minus the radii of its endpoint obstacles. You can thread dozens of nets through the same triangle edge simultaneously without committing them to coordinates, provided the sum of their widths and clearances does not exceed the edge's capacity.
- **Geometric Realization:** Only at the very end does the router perform a physical realization pass, pulling the topological paths taut like physical rubber bands (producing shortest-path Euclidean arcs and straight tangents) or snapping them to 45°/90° octilinear segments while pushing spacing outward to satisfy DRC.

2. Multi-Net Conflict Resolution Strategies

Because routing nets sequentially causes early nets to block later ones (the **net-ordering problem**), every modern autorouter relies on an outer loop to resolve resource contention.

Sequential Rip-Up and Reroute (R&R)

Used in classic PCB autorouters (e.g., Freerouting, Specctra):

1. Route nets sequentially based on a heuristic priority (usually shortest bounding-box perimeter first, or critical differential pairs first).
2. When Net k cannot reach its target without violating DRC, allow it to route *through* existing blocking traces at a massive penalty cost.
3. Add the intersected traces to a **rip-up queue**, delete them from the board, and attempt to reroute them around Net k.
4. **Failure Mode:** Sequential R&R is prone to **limit-cycle oscillations** (Net A rips up Net B, which rips up Net C, which rips up Net A). Routers combat this by randomizing net order on subsequent passes or inflating a local penalty map where rip-ups occur.

Negotiated Congestion (PathFinder)

Introduced by McMurchie and Ebeling (1995) for FPGAs and now standard across VLSI and advanced PCB global routers, **PathFinder** eliminates net-ordering bias by allowing all nets to overlap illegally at first, then iteratively pricing them out of congested regions until a legal solution emerges.

During iteration i, every routing resource node n (a grid cell, channel, or Delaunay edge) is assigned a dynamic cost:

$$c_n^{(i)} = \left(b_n + h_n^{(i)}\right) \cdot p_n^{(i)}$$

- b_n **(Base Cost):** The intrinsic geometric cost of using node n (trace length, layer preference, or via delay).
- $p_n^{(i)}$ **(Present Congestion Penalty):** A function of the current occupancy o_n vs. capacity C_n (e.g., $1 + \max(0, o_n - C_n) \cdot \alpha_p$). In iteration 1, $\alpha_p = 0$, so every net takes its selfish shortest path regardless of shorts. Over subsequent iterations, α_p ramps up, making shared nodes expensive.
- $h_n^{(i)}$ **(Historical Congestion Penalty):** Updated at the end of each full pass:

$$h_n^{(i+1)} = h_n^{(i)} + \beta_h \cdot \max(0, o_n - C_n)$$

Why h_n is the secret sauce: Without h_n, two nets fighting over a bottleneck channel will endlessly ping-pong back and forth on alternating iterations. Because h_n permanently ratchets up the price of a historically contested node even during iterations where one net temporarily vacates it, the router forces the net with the lower opportunity cost (the one with an easier alternative detour) to permanently abandon the bottleneck.

Exact Combinatorial Solvers (SAT, SMT, and ILP)

Instead of routing one net at a time, **Boolean Satisfiability (SAT)**, **Satisfiability Modulo Theories (SMT)**, and **Integer Linear Programming (ILP)** routers formulate a region of the board as a single giant system of equations and solve all nets simultaneously:

- **Variables:** Binary indicator $x_{e,k} \in \{0, 1\}$ denoting whether net k uses graph edge e.
- **Constraints:** Flow conservation at every node ($\sum_{\text{in}} x - \sum_{\text{out}} x = 0$), capacity exclusivity ($\sum_k x_{e,k} \leq 1$), and geometric spacing exclusion clauses.
- **Practical Tradeoff:** Because ILP/SMT scales exponentially with grid size and net count, it cannot route a whole PCB. Instead, modern routers use ILP/SAT strictly for **BGA escape routing (pin-breakout)**, **channel switchboxes**, and **layer-via assignment**, where finding a mathematically proven conflict-free breakout for 400 densely packed pins in a $2\ \text{cm} \times 2\ \text{cm}$ window is far more valuable than global speed.

3. Hierarchical Pipeline Architecture

To bridge global topology with micron-level manufacturing rules, industrial autorouters (especially in ASIC/VLSI and high-density interconnect PCBs) split execution into a 4-stage pipeline:

1. **Pin Escape / Fanout Generation:** Before routing point-to-point nets, the router solves the local geometry under fine-pitch BGAs and QFN packages, placing dog-bone or via-in-pad microvias to create accessible escape channels on inner layers.
2. **Global Routing (Coarse G-Cells):** The board is divided into a coarse grid of **Global Routing Cells (G-Cells)** or topological super-channels. The global router uses **Rectilinear Steiner Minimum Trees (RSMT)** (such as the **FLUTE** lookup-table algorithm) combined with Negotiated Congestion to assign each net to a corridor of G-Cells and specific copper layers, balancing thermal/copper density and avoiding global bottlenecks without calculating exact (x, y) trace coordinates.
3. **Detailed Routing (Track Assignment & DRC):** Operating inside the corridors defined by the global router, a shape-based or high-resolution A^* router resolves exact copper geometry, clearance rules, pad entry angles, and via-to-via spacing.
4. **Post-Route Optimization ("Glossing" & Push-and-Shove):** Raw autorouted traces look jagged ("stair-stepped"). A geometric **glossing engine** runs a rubber-band relaxation pass: it deletes unnecessary corners, converts 90° bends to 45° miters or arcs, rips up redundant vias, and centers traces between obstacles to maximize manufacturing yield.

4. Machine Learning & AI-Driven Routing (Current State of the Art)

Recent commercial and academic systems (e.g., Quilter, DeepPCB, Cadence Allegro X AI, Zuken Mimic, and academic RL frameworks) have shifted away from hand-tuned heuristics toward three distinct AI architectures:

- **Reinforcement Learning (RL) + Monte Carlo Tree Search (MCTS):** Treats PCB routing as a sequential Markov Decision Process (MDP) analogous to AlphaZero. Rather than relying on a static heuristic for net ordering and rip-up selection, a policy network (often a Graph Neural Network or CNN operating on G-Cell congestion maps) learns *which* net to route or rip up next to maximize long-term completion without entering limit cycles.
- **Joint Placement & Routing (Co-Optimization):** A fundamental flaw of traditional autorouters is treating component placement as an immutable input—if a decoupler or connector is rotated poorly, no routing algorithm can produce a clean layout. Physics-driven AI layout engines optimize placement and global routing concurrently across thousands of parallel cloud rollouts, scoring candidates via fast **Physics Rule Checks (PRCs)** (estimating loop inductance, return-path discontinuities, and PDN impedance) before committing to detailed geometry.
- **Agentic DRC-Repair Loops:** Emerging CAD workflows pair a fast deterministic shape-based router with a vision/language agent that inspects the resulting DRC violation clusters, dynamically adjusts local net-class clearances, injects strategic keep-out zones to force bus grouping, and re-invokes the deterministic solver iteratively.

5. Why Autorouters Still Struggle with Analog, Power, and High-Speed Layout

Even with negotiated congestion and shape-based geometry, algorithmic routers routinely produce layouts that pass 100% of geometric Design Rule Checks (DRC) while failing catastrophically on the bench. The gap lies in **implicit electromagnetic physics** that cannot be expressed as simple distance metrics:

- **Return Path & Reference Plane Discontinuities:** An A^* or shape-based router views a ground plane split or a dense row of antipads (via clearances in an inner ground plane) as free routing space on the adjacent signal layer. It does not inherently compute the 3D return current path (I_{return} following the path of least inductance directly beneath the trace) unless coupled to a 2.5D electromagnetic field solver, leading to massive radiated EMI loops when a trace crosses a reference split.
- **High-di/dt Switching Loops (SMPS / Gate Drives):** In a buck/boost converter or high-voltage gate driver, minimizing the parasitic inductance L_{loop} of the hot switching loop requires wide, overlapping copper polygons (Kelvin-connected or flux-canceling) rather than thin point-to-point traces. Standard autorouters model connections as 1D graph edges with a scalar width, making them blind to 2D polygon current crowding and skin-effect impedance.

- **Differential Pair Phase & Trombone Coupling:** While modern routers support coupled differential pair routing and serpentine length-tuning ("trombones"), their geometric engines often place length-matching meanders too close together (causing parasitic broadside/edge-to-edge self-coupling that lowers odd-mode impedance and causes signal edges to jump across the meander turns) or fail to compensate for static vs. dynamic phase skew around asymmetric pad breakouts.

Sources

1. How AI PCB Autorouting Works, and How to Check the Result
2. PathFinder FPGA Routing Algorithm – CADApps - Sites at Lafayette
3. Routing (electronic design automation) - Wikipedia
4. Automated PCB Routing: Autorouters vs. AI (2026 Guide) - Quilter AI

User prompt: Tell me more about the state of the art. I want deep research into how and why they do those things so I can have an LLM implement those features for my own personal use.

Response: Building a modern, state-of-the-art (SOTA) autorouter from scratch—especially via an LLM coding workflow—requires a critical architectural realization up front: **modern "AI/SOTA" autorouters do not replace deterministic geometry engines with neural networks; they wrap high-speed deterministic engines inside smarter search, pricing, and physics-scoring loops.**

If you ask an LLM to build an autorouter directly on a fine uniform grid, it will hit an $\mathcal{O}(W \cdot H \cdot L)$ memory wall on your first real PCB. To build a personal SOTA router (suitable for parsing KiCad `.kicad_pcb` or Specctra `.dsn`/`.ses` files), you want to architect it as **six decoupled, mathematically rigorous subsystems** that an LLM can implement module-by-module in Rust or C++ (with Python bindings for orchestration).

1. The Spatial Substrate: Dynamic Polygon Partitioning & Implicit Graphs

Why SOTA Does This

Classic routers fail because of the **Global-to-Detailed Routing Gap**: a coarse Global Router thinks two traces can fit between two pads, but when the Detailed Router tries to place the actual copper with net-class clearances and chamfered corners, it fails DRC. Conversely, pure gridless shape-based routers struggle to evaluate "congestion" because continuous empty space has no discrete edges to attach congestion prices to.

The SOTA solution (used in modern academic routers and commercial topological/hybrid engines) is **Dynamic Routable Region Partitioning** via **Constrained Delaunay Triangulation (CDT)** or **Trapezoidal Cell Decomposition**.

How It Works Under the Hood

Instead of a pixel grid, represent every obstacle on layer l (pads, static vias, keepouts) as a 2D polygon.

1. **Constrained Delaunay Triangulation (CDT):** Triangulate the vertices of all obstacle polygons on each layer. Every triangle T_i represents a convex region of free space; every triangle edge e_{ij} connecting two obstacles O_a and O_b acts as a **routing portal (channel)**.

2. **Exact Portal Capacity Calculation:** The maximum copper capacity $C(e_{ij})$ across a triangle edge between obstacle polygons O_a and O_b is the Euclidean distance between the closest points of the two polygons:

$$C(e_{ij}) = \min_{p \in O_a, q \in O_b} \|p - q\|_2$$

   When nets $k \in \text{Nets}(e_{ij})$ pass through portal e_{ij}, the exact required width is:

$$D(e_{ij}) = \sum_{k} w_k + \max(c_{\text{oa}}, c_{k_1}) + \max(c_{\text{ob}}, c_{k_m}) + \sum_{m=1}^{|\text{Nets}|-1} \max(c_{k_m}, c_{k_{m+1}})$$

   where w_k is trace width and c_k is the net-class clearance. **Why this matters:** Checking $D(e_{ij}) \leq C(e_{ij})$ guarantees in $\mathcal{O}(1)$ time during global search that a detailed geometric route *provably exists* through that gap, regardless of mixed metric/imperial pad pitches or varying high-voltage vs. logic clearance rules.

3. **3D Via Portals:** Project candidate via locations at the centroids (or in-centers) of the 2D triangles across layers, linking layer l to layer $l \pm 1$ in a sparse 3D graph $G = (V, E)$ with only thousands of nodes instead of hundreds of millions of voxels.

2. Multi-Pin Tree Construction: Obstacle-Avoiding RSMT (FLUTE + PD-II)

Why SOTA Does This

Point-to-point A^* only connects two pins. Real PCB nets (especially ground, power rails, SPI buses, or address lines) have $k \geq 3$ pins. Connecting k pins sequentially via Minimum Spanning Tree (MST) produces awkward "V"-shaped junctions at pads rather than clean T-junctions (Steiner points) in mid-air, increasing total wirelength by 12–15% and hogging pad escape channels.

How It Works Under the Hood

SOTA routers decompose multi-pin nets into two-pin subnets (or route directly to existing copper) using two algorithms:

1. **FLUTE (Fast Lookup Table Based RSMT) / GeoSteiner:** For nets with $k \leq 9$ pins, pre-computed lookup tables grouped by pin rank ordering (called *PowVs*—Potentially Optimal Wirelength Vectors) compute the exact **Rectilinear Steiner Minimum Tree** in $\mathcal{O}(k \log k)$ time.
2. **PD-II (Prim-Dijkstra Tradeoff) for High-Speed Nets:** Standard Steiner trees minimize total wirelength, which can create terrible **path-length skew** (longest path from driver to receiver) on clock or memory buses. PD-II modifies Prim's algorithm with a trade-off parameter $\alpha \in [0, 1]$:

$$\text{Cost}(u, v) = d(u, v) + \alpha \cdot \text{PathDelay}(\text{Driver} \to u)$$

   - At $\alpha = 0$, it yields a minimum-wirelength tree (lowest capacitance).
   - At $\alpha = 1$, it yields a Shortest-Path Tree from the driver (minimum RC/transmission delay and skew).
3. **Dynamic Multi-Target Wavefront (A^* to Existing Subtree):** During detailed routing, once the first two pins of a net are connected by a trace made of segments $\{S_1, S_2, \ldots, S_m\}$, the entire set of segments becomes the target for the 3rd pin. The A^* heuristic $h(n)$ is evaluated as the minimum distance from node n to the bounding-box R-tree of all already-routed segments of that net, naturally forming T-junctions anywhere along the trace.

3. Core Conflict Resolution: Enhanced PathFinder with Directional & Via Pricing

Why SOTA Does This

Standard sequential rip-up-and-reroute gets stuck in local optima because early nets permanently block later ones. SOTA routers (including modern FPGA, ASIC, and cloud PCB engines) rely on **Negotiated Congestion (PathFinder)** with three critical PCB-specific modifications: **adaptive historical schedules**, **wrong-way directional biasing**, and **via-shadow congestion pricing.**

How It Works Under the Hood

Instead of banning overlaps, allow all nets to route through the same portals/cells initially, then run an outer loop of $i = 1 \ldots N_{\max}$ iterations.

1. **The Exact Cost Function for Edge e on Layer l:**

$$\text{Cost}(e, k, i) = \left(b_e \cdot \psi_{\text{dir}}(e, l) + h_e^{(i)}\right) \cdot p_e^{(i)} + \Phi_{\text{physics}}(e, k)$$

   - **Base Cost (b_e):** Euclidean length of edge e, or equivalent via cost C_{via} (typically weighted equal to 2.5 mm–5.0 mm of trace length so the router prefers small detours over dropping layer-transition vias).
   - **Directional Layer Bias (ψ_{dir}):** To prevent a single trace from cutting across an entire layer and blocking 50 orthogonal signals, assign alternating preferred directions to signal layers (e.g., Layer 1 = Horizontal, Layer 2 = Vertical).

$$\psi_{\text{dir}}(e, l) = 1.0 + \lambda_{\text{bias}} \cdot \sin^2(\theta_e - \theta_{\text{pref},l})$$

     where $\lambda_{\text{bias}} \approx 0.5$–$1.5$. Traces can still route diagonally or orthogonally to the preferred direction near pins, but long runs naturally segregate by layer.

   - **Present Congestion Penalty ($p_e^{(i)}$):** Exponentially or linearly scales with over-utilization:

$$p_e^{(i)} = 1 + \alpha_p^{(i)} \cdot \max\left(0, \frac{D(e) - C(e)}{w_{\min}}\right)$$

     Start with $\alpha_p^{(1)} = 0.2$ and multiply by $1.3\times$ each iteration (a **sigmoid/exponential cooling schedule**).

   - **Historical Congestion Ratchet ($h_e^{(i)}$):** Updated after each iteration i:

$$h_e^{(i)} = h_e^{(i-1)} + \beta_h \cdot \max\left(0, \frac{D(e) - C(e)}{w_{\min}}\right)$$

Set $\beta_h \approx 0.5$–1.0. Crucially, apply a **decay factor** ($\gamma = 0.98$) to h_e if an edge has been legal ($D(e) \leq C(e)$) for 5 consecutive iterations so early transient traffic jams don't permanently warp late-stage routing.

2. **Via-Shadow (Antipad) Pricing:** On a PCB, a through-hole via doesn't just consume space on the two layers it connects; it punches a hole through **every layer in the stackup**, plus an antipad clearance on internal power/ground planes. When pricing a via edge e_{via}, its congestion penalty must sum the demand across *all* penetrated layers:

$$p_{\text{via}}^{(i)} = \sum_{l=l_{\text{top}}}^{l_{\text{bot}}} p_{e,l}^{(i)} + \omega_{\text{plane_split}} \cdot \mathbb{I}(\text{creates overlapping antipad slot})$$

That final term $\omega_{\text{plane_split}}$ penalizes placing vias so close together that their antipad clearances merge into a continuous slot in an internal reference plane.

4. Exact Combinatorial Escape Routing: SAT/SMT Pin Breakout

Why SOTA Does This

Negotiated congestion works brilliantly in open board channels, but inside a dense BGA, QFN, or high-density connector footprint, space is so tightly quantized that heuristic search thrashing often fails to find the unique topological permutation that lets all inner rows escape without crossing.

How It Works Under the Hood

SOTA engines isolate dense component footprints into a local bounding box and formulate **Pin Escape (Breakout)** as a **Boolean Satisfiability (SAT)** or **Integer Linear Programming (ILP)** problem before running the global router:

1. Define a local escape grid around the component pads to the perimeter of the component's bounding box.
2. Formulate as **Maximum Vertex-Disjoint Paths** (if unordered escape) or **Ordered Escape** (if the target component's pin order is already known, preventing crossovers in the channel):
   - Create boolean variables $v_{p,n,l}$ ("Net from pin p occupies grid node n on layer l").
   - **At-Most-One (AMO) Clauses:** For every node n, l, at most one net p can be true (encoded efficiently using **Sinz Sequential Counters** in CNF).
   - **Flow Continuity Clauses:** If $v_{p,n,l}$ is true and n is not on the boundary, at least one adjacent neighbor $n' \in \text{Adj}(n)$ must have $v_{p,n',l}$ true.
3. Pass the CNF formula to an off-the-shelf modern solver (**Z3**, **CaDiCaL**, or **HiGHS**). Within 100–500 milliseconds, the solver either returns a mathematically proven 100% breakout assignment (including exact dog-bone via coordinates) or returns `UNSAT`—proving instantly that the footprint requires another routing layer or smaller via rules.

5. Physics-Driven Cost Terms (How Engines Like Quilter & Allegro X AI Work)

Why SOTA Does This

The biggest leap in SOTA PCB autorouters (such as Quilter's Physics-Driven AI and Cadence's Allegro X AI) is moving electrical verification *inside* the inner routing cost loop rather than running an external SI/PI simulator after the board is finished.

How to Implement Fast Physics Proxies (Φ_{physics})

You do not need a slow 3D finite-element solver inside your A^* loop. Instead, evaluate analytical closed-form physics proxies at $\mathcal{O}(1)$ cost per candidate step:

1. **Reference Plane Return-Path Continuity Check:**
   - Maintain a 2D spatial index (R-tree) of all copper pours and antipad cutouts on adjacent reference layers $l_{\text{ref}} = l \pm 1$.
   - When evaluating a candidate trace segment $S = (p_1, p_2)$ on signal layer l, check whether the projection of S onto l_{ref} is 100% contained within a continuous polygon of the reference net (e.g., `GND`).
   - If S crosses a split or an antipad void on l_{ref} of width d_{gap}, add a massive inductance penalty:

$$\Phi_{\text{return}}(S) = \kappa_{\text{EMI}} \cdot d_{\text{gap}} \cdot \left(\frac{1}{t_{\text{rise}}(k)} \right)$$

where $t_{\text{rise}}(k)$ is the net-class rise-time weight (high for clocks/PWM/gate drives, zero for static status LEDs).

2. **Loop Inductance Proxy for Power & Gate-Drive Nets:**
   - For critical 2-net pairs $(k_{\text{out}}, k_{\text{ret}})$—such as a buck converter switch node and its return, or a MOSFET gate drive and Kelvin source return—route them as a **coupled loop pair**.
   - Estimate loop inductance L_{loop} dynamically via the **Enclosed Area Proxy**:

$$L_{\text{loop}} \approx \mu_0 \cdot \frac{\text{Area}_{\text{enclosed}}(k_{\text{out}}, k_{\text{ret}})}{w_{\text{eff}}}$$

   If k_{out} and k_{ret} are routed broadside-coupled on adjacent layers l and $l+1$ with dielectric thickness h, the enclosed area drops to $h \times \text{Length}$, which is an order of magnitude smaller than routing them side-by-side on the same layer. Reward broadside or tightly coupled co-planar routing directly in the A^* heuristic.

3. **Crosstalk (Aggressor-Victim Coupling) Integral:**
   - Query the R-tree for parallel trace segments within $3 \times H_{\text{dielectric}}$ of candidate segment S.
   - Accumulate a coupling score proportional to parallel run length L_{par} divided by edge-to-edge spacing squared:

$$\Phi_{\text{xtalk}}(S) = \sum_{j \in \text{Neighbors}} \gamma_{k,j} \cdot \frac{L_{\text{par}}(S,j)}{s(S,j)^2}$$

   where $\gamma_{k,j}$ is high when routing an aggressor (e.g., switching node or clock) near an analog/high-impedance victim.

6. ML / RL Search Orchestration (MCTS + Policy Rollouts)

Why SOTA Does This

Even with PathFinder, tough boards (high pin density, 4–6 layers, tight clearances) hit combinatorial dead-ends depending on **hyperparameter schedules** and **rip-up prioritization.** SOTA AI routers (like DeepPCB, Quilter, and academic MCTS routers) treat the routing engine as a **Markov Decision Process (MDP)** and wrap it in **Monte Carlo Tree Search (MCTS)** or **Parallel Evolutionary Rollouts.**

How It Works (And How to Implement It Without Training a Massive Model)

You do not need a cluster of 1,000 GPUs or 40,000 proprietary training boards to get 90% of the benefit of AI routing. The secret is **MCTS + UCB1 (Upper Confidence Bound)** over the Rip-Up and Net-Ordering state space:

1. **State (s_t):** A partial or congested board layout where M nets are legally routed and U nets have DRC violations or unrouted airwires.
2. **Action (a_t):** Selecting a specific subset of conflicting obstacle nets $\{k_1, k_2\}$ to rip up, inflating their local obstacle penalty weights β_e, or permuting the priority order of a net class (or flipping a passive component 180° to untangle crossed airwires).
3. **Tree Search Policy (UCB1):** Instead of blindly repeating PathFinder when it stalls at 97% completion, branch into a search tree of board states, selecting branches via:

$$\text{UCB1}(s,a) = Q(s,a) + c_{\text{puct}} \cdot P(s,a) \cdot \frac{N(s)}{1+N(s,a)}$$

   - $Q(s,a)$ is the best completion score ($-\text{DRC_violations} - \alpha \cdot \text{Total_Wirelength} - \beta \cdot N_{\text{vias}}$) achieved from that subtree.
   - $P(s,a)$ is a fast prior probability: prioritize actions that rip up the *longest* blocking net that crosses the *most* unrouted bounding boxes (or use a lightweight GNN/MLP if you later want to train one on your own boards).

7. Geometric Realization & Push-and-Shove Glossing

Why SOTA Does This

Raw output from A^* or topological channel routing looks robotic: stair-stepped 45° zig-zags, acute acid traps, and traces hugging obstacles at minimum clearance even when the rest of the channel is empty.

How It Works Under the Hood

The final stage of a SOTA router is a **Rubber-Band + Octilinear Glossing Pass**:

1. **Shortcut & Corner Deletion:** Iterate over every 3-segment sliding window (S_{m-1}, S_m, S_{m+1}) of a routed trace. Construct a direct candidate segment or 45° miter connecting S_{m-1} to S_{m+1}, query the R-tree for expanded obstacle collisions, and if clear, replace the three segments with the shorter path. Repeat until convergence.
2. **Differential Pair Coupling & Serpentine (Trombone) Tuning:**
   - Route differential pairs as a single **virtual centerpiece ribbon** of width $W_{\text{ribbon}} = 2w_{\text{trace}} + s_{\text{diff}}$ through the global and detailed router.
   - Unpack the centerline into two parallel offset polylines at the very end, adding symmetric phase-matching bumps near the pad breakouts where pin pitch doesn't match $w_{\text{trace}} + s_{\text{diff}}$.
   - For length-matching meanders, enforce a minimum self-spacing rule of $s_{\text{meander}} \geq 3.5 \times H_{\text{dielectric}}$ (or $4 \times w_{\text{trace}}$) inside the trombone generator so high-speed edges don't cross-couple across the turns.
3. **Via Elimination & Centering:** Check every pair of vias on the same net; if the trace segments between them can be moved to the adjacent layer without violating DRC, delete both vias and merge the trace. Finally, inflate clearances iteratively ($1.0\times \rightarrow 1.5\times \rightarrow 2.0 \times C_{\text{rule}}$) on non-impedance-controlled nets to push traces into the centers of open channels for better manufacturing yield.

8. Blueprint for Having an LLM Build This for You

If you tell an LLM "write me a SOTA PCB autorouter," it will write a 300-line Python script with a slow 2D NumPy grid and A^*. To actually get a high-performance, SOTA-grade engine built via an LLM (such as Claude Code, Cursor, or Gemini CLI), have it build a **Rust workspace** (or C++20) structured around the industry-standard **Specctra `.dsn` (input) / `.ses` (output)** or **KiCad S-expression (`.kicad_pcb`)** format so it plugs directly into KiCad:

Module / Milestone	Key Libraries to Instruct the LLM to Use	Core Responsibility
1. I/O & Geometry Kernel	`sexp` (KiCad parser), `geo` / `cavalier_contours` (2D polygons & offsets), `rstar` (R-tree spatial index)	Parse `.kicad_pcb` or `.dsn` into exact 2D obstacle polygons per layer; implement Minkowski expansion ($O \oplus r$) and exact segment-to-polygon clearance queries in $\mathcal{O}(\log N)$.
2. CDT & Portal Graph	`spade` (Constrained Delaunay Triangulation in Rust)	Build the per-layer CDT mesh from pad/obstacle vertices; compute exact portal capacities $C(e_{ij})$ and construct the 3D sparse routing graph.
3. Multi-Pin Tree & SAT Escape	`varisat` or `good_lp` (SAT/ILP solver)	Implement Prim-Dijkstra (PD-II) Steiner decomposition for multi-pin nets and SAT-based pin escape for dense QFN/BGA footprints.
4. Negotiated Congestion Engine	`petgraph` / custom binary heap A^* + `rayon` (parallelism)	Implement the PathFinder outer loop with directional layer bias ψ_{dir}, historical congestion h_e, and via-shadow antipad pricing.
5. Physics Cost Plugins	Custom R-tree plane-projection queries	Add modular cost hooks Φ_{return} (reference plane split penalty) and Φ_{xtalk} (parallel aggressor coupling penalty) into the A^* edge evaluator.
6. Geometric Realizer & Glosser	`cavalier_contours` / custom octilinear fitter	Convert portal sequences into exact 45°/90° or arc traces; run shortcut corner elimination, via removal, and differential pair unpacking.

Why Rust + `.kicad_pcb` / `.dsn` is the Ideal Target

- **Zero GUI Overhead:** By reading and writing `.kicad_pcb` (or `.dsn`/`.ses`), KiCad remains your viewer, schematic editor, and final DRC verifier. Your custom router runs as a fast CLI tool (`my-router board.kicad_pcb --out routed.kicad_pcb`).
- **Deterministic Testability:** You can have the LLM write unit tests for every single stage in isolation (e.g., testing that Minkowski clearance queries match KiCad's DRC to within $1\ \mu\text{m}$, or testing that the PathFinder loop resolves a synthetic 20-net switchbox crossover in under 50 ms) before assembling the full pipeline.

Sources

1. What Is the Best AI PCB Autorouter in 2026? - DeepPCB.ai

User prompt: Does any of this cover part placement? Could you dive more into that? And then also, if there's any kinds of greater loops involved with especially LLMs in the loop where they might revisit part placement when they start hitting issues.

Response: The previous breakdown treated component coordinates as fixed inputs. In reality, **80% of routing failures are actually placement failures in disguise.** If a decoupling capacitor is placed on the wrong side of an IC, or an MCU is rotated 90° counter-clockwise relative to a connector, no routing algorithm on earth can untangle the resulting "rat's nest" without dropping dozens of vias and destroying signal integrity.

Modern SOTA tools (like Quilter, Cadence Allegro X AI, and academic frameworks like Google's Circuit Training and Instacart/UC San Diego's PCB-Place) solve **Placement and Routing as a coupled, bi-level optimization loop.** Furthermore, the newest paradigm in EDA is **Agentic / LLM-in-the-loop Eco-Routing**, where an LLM acts as a "Lead Layout Architect" that reads routing congestion reports, diagnoses *why* the router failed, and mutates the placement or schematic pin-swapping to break the deadlock.

Here is the deep dive into how SOTA placement works mathematically, how routability-driven feedback loops operate, and how to architect an LLM-in-the-loop placement-and-routing system.

1. Why PCB Placement is Fundamentally Harder Than VLSI/Chip Placement

In chip design (VLSI), placement algorithms place millions of uniform-height standard cells into neat horizontal rows. **PCB placement is drastically more irregular and constrained:**

1. **Extreme Size Disparity ("Boulders and Dust"):** You are simultaneously placing a $40\text{ mm} \times 40\text{ mm}$ connector or heatsink alongside 0402 ($1.0\text{ mm} \times 0.5\text{ mm}$) passives.
2. **Continuous Rotation & Double-Sided Assembly:** Components can rotate $\theta \in \{0^\circ, 90^\circ, 180^\circ, 270^\circ\}$ (or continuous $\theta \in [0, 2\pi)$) and can be flipped between the Top and Bottom layers ($z \in \{0, 1\}$), which mirrors their pin coordinates across the X/Y axis.
3. **Strict Physics & Clustering Rules:** A buck converter's high-side FET, low-side FET, inductor, and input capacitor *must* form a tight geometric cluster to minimize switching loop inductance, regardless of where the rest of the board's signals pull them.

Because of this, SOTA PCB placement is broken into **three distinct mathematical paradigms**.

2. The Three SOTA Placement Paradigms

Paradigm	Core Representation	Strengths	Failure Modes	Best Used For
1. Topological Packing + Simulated Annealing	*B-Trees** or **Sequence Pairs**	Never produces overlapping parts; handles huge size differences ("boulders & dust") natively	Slow convergence on > 500 parts; blind to exact continuous coordinates until unpacked	Small-to-medium analog/power/MCU boards (< 300 parts); local functional clusters
2. Analytical / Electrostatic (ePlace / RePlAce)	Continuous differentiable field + Electrostatics	Scales to $10,000+$ components in seconds via GPU gradient descent	Struggles with discrete 90° rotations and hard keepout zones without heavy legalization	Large digital boards, backplanes, FPGA carrier boards
3. RL / Graph Neural Network Sequential Placement	MDP state graph (Pins/Nets = Graph, Board = Action Grid)	Learns non-obvious topological foresight and routability patterns	Requires pre-training or hundreds of rollout episodes	Placing the 10–30 "core" macro ICs before analytical/SA passives placement

Paradigm A: Topological Representation (Sequence Pairs & B*-Trees) + Simulated Annealing

If you let an optimizer randomly nudge (x, y) coordinates of components, 99% of moves result in illegal component overlaps. SOTA discrete placers never search (x, y) space directly; they search **topological packing space** using a **Sequence Pair** (Γ_+, Γ_-).

- **How Sequence Pairs Work:** Given N rectangular components, a state is just two permutations of component IDs: Γ_+ and Γ_-, plus a orientation vector $\Theta \in \{0, 1, 2, 3\}^N$ and layer vector $L \in \{\text{Top}, \text{Bot}\}^N$.
  - If component A appears before B in *both* Γ_+ and Γ_-: A **is strictly to the left of** B ($x_A + w_A \le x_B$).
  - If A appears before B in Γ_+, but *after* B in Γ_-: A **is strictly above** B ($y_A \ge y_B + h_B$).
- **Why This is Magic:** Every possible permutation of (Γ_+, Γ_-) can be unpacked into exact (x, y) coordinates in $\mathcal{O}(N \log N)$ time using a Longest Common Subsequence (LCS) graph algorithm, and **it is mathematically guaranteed to be 100% overlap-free.**
- **The Search Loop:** You run **Simulated Annealing (SA)** or **Parallel Tempering** by randomly swapping two items in Γ_+, rotating a part 90°, or flipping a passive to the bottom layer, unpacking the coordinates, and scoring the layout.

Paradigm B: Analytical / Electrostatic Placement (ePlace / DreamPlace Adapted for PCBs)

How do SOTA tools place 1,000 components simultaneously while minimizing wirelength and rotation? They treat components as **positively charged particles in an electrostatic field** and nets as **elastic springs**:

$$\min_{\mathbf{x},\mathbf{y},\boldsymbol{\theta}} \mathcal{L}(\mathbf{x},\mathbf{y},\boldsymbol{\theta}) = \mathrm{WA}(\mathbf{x},\mathbf{y},\boldsymbol{\theta}) + \lambda \cdot \Phi_{\text{electrostatic}}(\mathbf{x},\mathbf{y}) + \eta \cdot \sum_i \sin^2(2\theta_i)$$

1. **Weighted-Average (WA) Smooth Wirelength:** Actual bounding-box wirelength (HPWL) uses $\max(x_i) - \min(x_i)$, which is non-differentiable. SOTA placers replace $\max$ with the smooth differentiable approximation:

$$\mathrm{WA}_x(e) = \frac{\sum_{i\in e} x_i \exp(x_i/\gamma)}{\sum_{i\in e} \exp(x_i/\gamma)} - \frac{\sum_{i\in e} x_i \exp(-x_i/\gamma)}{\sum_{i\in e} \exp(-x_i/\gamma)}$$

2. **Continuous Pin Rotation:** A pin with local offset (u_p, v_p) from component center (x_i, y_i) has global coordinates:

$$X_p = x_i + u_p \cos\theta_i - v_p \sin\theta_i, \quad Y_p = y_i + u_p \sin\theta_i + v_p \cos\theta_i$$

   Because $\sin\theta_i$ and $\cos\theta_i$ are smooth and differentiable, you can compute exact analytical gradients $\frac{\partial \mathcal{L}}{\partial \theta_i}$! As the optimizer runs **Nesterov Accelerated Gradient Descent**, the nets literally apply physical torque to each IC, smoothly spinning it to untwist its airwires while the $\sin^2(2\theta_i)$ penalty gently snaps it into a $0^\circ, 90^\circ, 180^\circ$, or 270° well at the end.

3. **Electrostatic Repulsion ($\Phi_{\text{electrostatic}}$):** Component density is mapped onto a 2D grid, and Poisson's equation ($\nabla^2\psi = -\rho$) is solved via a **2D Fast Fourier Transform (FFT)** in milliseconds. Overlapping components generate high electric potential ψ, pushing each other apart into empty board regions.

Paradigm C: Hierarchical Functional Clustering ("Satellite" Decouplers & Power Stages)

No SOTA PCB placer dumps raw passives into the global optimizer alongside large ICs. Instead, they use **Hierarchical Pre-Clustering**:

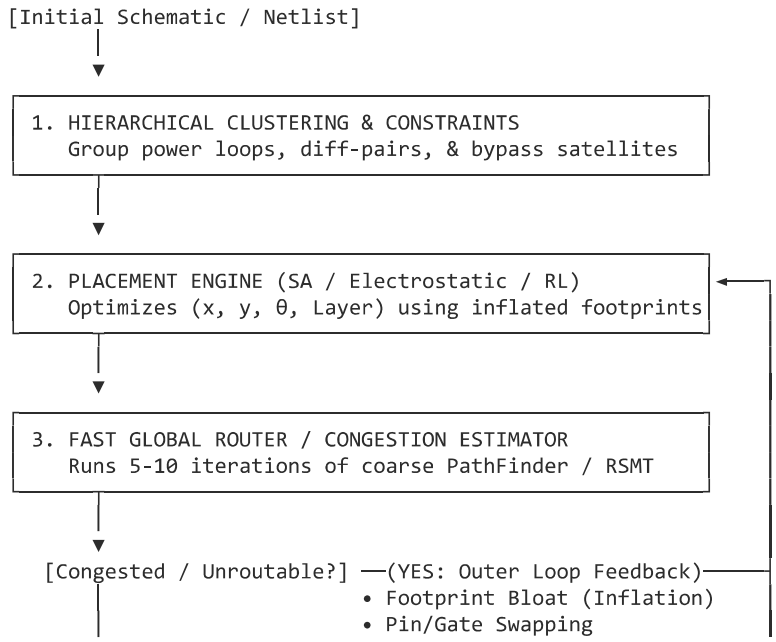
1. **Anchor & Satellites:** Before placement begins, a graph analyzer (or an LLM reading the schematic) binds every bypass capacitor, pull-up resistor, and crystal load cap as a **"Satellite"** of its parent IC pin, and binds switching power stages (FET + Inductor + Cin + Cout) into rigid or semi-rigid **Functional Macros.**

2. During global placement, the parent IC moves with an inflated courtyard that reserves space for its satellites. Once the parent IC locks into an (x, y, θ), a local sub-placer packs the satellites directly adjacent to their target power/signal pins (or directly beneath the IC on the Bottom layer via through-vias).

3. The Greater Loop: Routability-Driven Placement & Co-Optimization

Here is where placement and routing merge. Minimizing wirelength (HPWL) alone is a trap: **minimum wirelength pulls all components into a tight clump in the center of the board, creating an unroutable traffic jam.**

SOTA engines use a **Closed-Loop Placement-Routing Feedback Cycle**:



```
      |                    • LLM Structural Diagnosis       |
      ▼ (NO: 100% Routable)

+-----------------------------------------------------------+
| 4. DETAILED ROUTING, PUSH-AND-SHOVE & GLOSSING            |
```

Mechanism 1: Congestion-Driven Footprint Inflation (Cell Bloat)

When the Global Router (or early PathFinder iterations) fails to resolve congestion in a specific region R of the board (i.e., portal demand $D(e) > C(e)$):

1. Do **not** throw away the placement and start over.
2. Compute the local congestion ratio $\gamma_R = \frac{D(R)}{C(R)}$ in the congested triangles/G-cells.
3. **Artificially inflate the courtyard polygon** of every component sitting inside or bordering region R by a factor proportional to γ_R (e.g., bloating a QFN's courtyard by $1.5\times$ on its congested East edge).
4. Re-run the Placement Legalizer / Simulated Annealer for a brief warm-start pass. The inflated courtyards physically shove the neighboring components apart, opening up the exact millimeter of copper channel the router was starving for.

Mechanism 2: Routing-Driven Pin, Gate, and Sub-Net Swapping

Often a board fails to route solely because four GPIO traces from an STM32 or FPGA cross over each other on the way to a header or memory chip.

- **Dynamic Bipartite Matching inside PathFinder:** If a pin group $\{p_1, p_2, \dots, p_m\}$ is marked as **swappable** (e.g., general-purpose MCU timer pins, resistor array elements, or SDRAM DQ bits within a byte lane), the router does not hardwire Net k to Pin p_k.
- Instead, during iteration i of PathFinder, the router computes a **Minimum-Weight Bipartite Matching (Hungarian Algorithm)** between the net targets and the swappable pin group based on current congestion distances. This dynamically uncrosses entire buses in $\mathcal{O}(m^3)$ time without moving a single physical part.

4. LLM-in-the-Loop Architecture: How to Build an Agentic Layout Engine

This is the frontier you can implement yourself right now, and it is uniquely suited to an LLM-driven architecture.

Why LLMs Fail at Raw Coordinates—And Where They Excel

If you prompt an LLM: *"Place U1 at (x, y) and route Net 1 from (12.4, 33.1) to (45.2, 18.9),"* it will hallucinate horribly. LLMs lack fine-grained Euclidean spatial precision.

However, **LLMs possess world-class semantic circuit intuition and topological diagnosis skills** that deterministic algorithms completely lack. A deterministic simulated annealer does not know what an `STM32G4`, a `TPS54302` buck converter, or a `CAN transceiver` is—it just sees rectangles and graph edges. An LLM knows immediately from the netlist and part numbers:

- Which parts form the hot switching loop of a power supply.
- Which passives are ESD diodes that must sit within $2\ \text{mm}$ of an external USB/CAN connector before the trace hits the MCU.
- Which differential pairs need symmetry.
- Why a specific component orientation is causing an unroutable bottleneck.

The 3-Tier Agentic Loop Architecture

To build this for your personal use, structure the LLM as a **Semantic & Diagnostic Supervisor** sitting on top of your fast deterministic Rust/C++ Placer and Router.

Loop 1: Pre-Placement Semantic Constraint Synthesis (LLM $\rightarrow$ Placer)

Before any math runs, pass the parsed KiCad schematic/netlist JSON to the LLM. The LLM outputs a structured **Layout Constraint Specification (JSON/DSL)** that configures the deterministic placer:

1. **Functional Hierarchy:** Groups parts into rigid/elastic rooms (e.g., `Room_Buck_5V: [U2, L1, C4, C5]`, `Room_USB_Frontend: [J1, D_ESD1, R_CC1, R_CC2]`).
2. **Relative Topological Anchors:** Emits Sequence-Pair or compass constraints:
   - `"D_ESD1" must be placed STRICTLY_BETWEEN("J1", "U1") with max_distance = 3.0mm`

- `"C_decouple_1" must be bound to "U1.Pin_VDD_1" with max_loop_area = 4.0mm^2`

3. **Layer & Net-Class Physics Rules:** Tags high-di/dt loops (`SW_NODE`), sensitive analog sense lines (`V_SENSE_POS/NEG` as pseudo-differential Kelvin pairs), and swappable MCU pins.

Loop 2: The Diagnostic ECO Loop (Router Failure $\rightarrow$ LLM Diagnosis $\rightarrow$ Placement Mutation)

When the deterministic PathFinder router hits a wall (e.g., after 30 iterations, 6 nets still have unresolvable DRC shorts or crossings), trigger the **LLM Diagnostic Loop**:

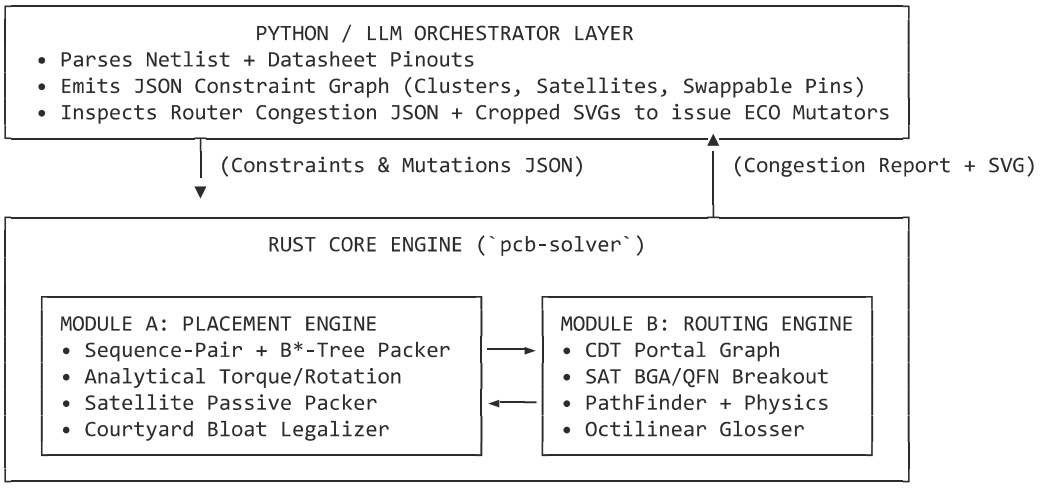
1. **Generate a Multi-Modal Failure Report:** Your Rust engine exports two things to the LLM:
   - **A Structured Text Bottleneck Report:**

     ```
     BOTTLENECK_CLUSTER_1 at bbox (x: 22.0..28.5, y: 14.0..19.0):
     - Over-congested portals: [ (U1.Pad_14 <-> C12.Pad_1): capacity 0.45mm, demand 0.90mm ]
     - Unrouted/Shorted Nets: [ SPI1_MOSI, SPI1_SCK, BUCK_SW ]
     - Blocking Components in BBox: C12 (0603, rot=90°, Top), R8 (0402, rot=0°, Top), U1 (LQFP-48, rot=0°, Top)
     - Airwire Crossings around U1: 11 crossings between U1 East edge and J2 West edge.
     ```

   - **A Cropped SVG / Raster Heatmap (Vision Input):** Render a high-contrast 2D image of the $15\ \text{mm} \times 15\ \text{mm}$ congested zone showing component outlines, pin numbers, routed copper, and glowing red congestion portals. Multimodal models (like Gemini 2.5 Pro or Claude) are remarkably good at looking at a cropped PCB congestion image and spotting the exact geometric trap.
2. **Restricted Action Space (DSL Tool Calls) for the LLM:** Never ask the LLM to output raw (x, y) coordinates for every part. Instead, give it a **Topological Mutation Toolset**:
   - `rotate_component(refdes="U1", delta_deg=90)` — *"U1 has 11 crossed airwires to J2; rotating U1 by 90° clockwise aligns the SPI bank directly with J2."*
   - `flip_to_bottom_layer(refdes=["C12", "C13", "R8"])` — *"C12 and R8 are blocking the SPI escape channel on the Top layer; move these decoupling/pull-up passives to the Bottom layer directly under U1."*
   - `inflate_courtyard(refdes="U1", side="EAST", extra_mm=1.5)` — *"Force the placer to clear a 1.5mm routing canal along the East edge of U1."*
   - `swap_relative_order(comp_a="U3", comp_b="U4", axis="Y")` — *"Swap the vertical stacking order of the two gate drivers so their inputs don't cross."*
   - `swap_mcu_pins(net_a="TIM1_CH1", net_b="TIM1_CH2")` — *"Uncross PWM traces at the MCU breakout."*
3. **Fast Re-Legalization & Incremental Re-Route:** When the LLM calls `flip_to_bottom_layer(["C12", "R8"])` and `inflate_courtyard("U1", "EAST", 1.5)`, your deterministic engine executes a 200-millisecond local placement legalization pass (sliding neighboring parts out of the way of the inflated courtyard without scrambling the rest of the board) and resumes PathFinder.

5. Concrete Blueprint: How to Architect Your Personal "AI Placer + Router"

If you want to have an LLM code this entire stack for your own boards, build it as a **3-Stage Modular Pipeline** operating on KiCad `.kicad_pcb` files:



Step-by-Step Implementation Plan for Your Coding LLM

1. **Phase 1: The Differentiable + Sequence-Pair Placer (`placer` crate)**
    - Have the LLM implement a **Sequence-Pair (Γ_+, Γ_-) unpacker** using a Longest Common Subsequence (LCS) algorithm in Rust. Because LCS unpacking for 150 components takes $< 50\ \mu s$ in Rust, a single CPU core can evaluate **20,000+ overlap-free placements per second** via Simulated Annealing.
    - Build the **Placement Cost Function**:

$$\text{Score} = w_1 \cdot \text{HPWL}_{\text{weighted}} + w_2 \cdot \text{AirwireCrossings} + w_3 \cdot \text{ClusterPenalty} + w_4 \cdot \text{BoardArea}$$

    - **Pro-Tip — The Airwire Crossing Counter:** Add a fast line-segment intersection counter over the unrouted rat's nest (airwires) directly into the placement score! Minimizing airwire crossings during placement is the single highest-leverage heuristic for making the subsequent routing pass trivial.
2. **Phase 2: Satellite Passive Binding**
    - Implement a rule where 2-pin passives (bypass caps, series termination resistors, ESD diodes) are excluded from the main Sequence-Pair permutation during macro placement, and are instead attached as **elastic springs** to their parent IC pins, snapping into the nearest legal courtyard slot on Top or Bottom once the macro ICs settle.
3. **Phase 3: Coupling the CDT Router to the Placer**
    - After Simulated Annealing converges on a placement, build the Constrained Delaunay Triangulation (CDT) and run 10 fast iterations of PathFinder.
    - For any CDT triangle edge where copper demand exceeds portal capacity ($D(e) > C(e)$), compute the expansion vector and automatically bloat the bounding boxes of the two obstacle components forming that edge, then run 1,000 warm-start Simulated Annealing steps to open the channel.
4. **Phase 4: The LLM Supervisor Loop (MCP Server or CLI Agent)**
    - Expose the Rust engine as a local **Model Context Protocol (MCP) server** or CLI tool with commands: `analyze_netlist`, `run_placement`, `run_router`, `render_congestion_svg`, and `apply_mutation`.
    - Now your coding LLM (Claude Code / Gemini CLI) doesn't just write the software—**it can actively drive the layout of your board**, iteratively viewing the congestion SVG, flipping passives, rotating ICs, adjusting net-class priorities, and re-running the Rust solver until KiCad's native `kicad-cli pcb drc` returns `0 violations, 0 unconnected items`.
